Les protocoles de la couche « application »

La couche « application » est la plus haute du modèle TCP/IP. Elle permet la communication directe entre les programmes utilisateurs (navigateur, client mail, messagerie instantanée, etc.).

Elle définit les protocoles applicatifs, c'est-à-dire les règles de dialogue entre applications sur deux machines.

Chaque protocole utilise un numéro de port pour être identifié par la couche transport (TCP/UDP).

Qu'est ce qu'un numéro de port ?

Un numéro de port est un identifiant logique (un nombre) utilisé par le système d'exploitation pour distinguer les applications qui communiquent sur le réseau.
→ Il sert à diriger les messages vers le bon programme à l'intérieur de ton ordinateur.

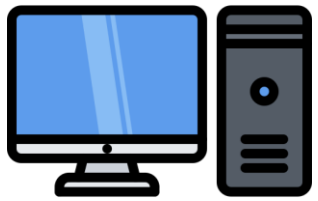


Les protocoles de la couche « application »

Protocole	Service assuré	Port par défaut	Fonction principale
HTTP / HTTPS	Web	80 / 443	Transfert de pages web
FTP	Transfert de fichiers	21	Envoyer / recevoir des fichiers
SMTP	Courrier électronique (envoi)	25	Envoi de mails
POP3 / IMAP	Courrier électronique (réception)	110 / 143	Récupérer les mails
DNS	Résolution de noms	53	Convertir un nom de domaine en adresse IP
DHCP	Attribution d'adresses IP	67-68	Donner dynamiquement une IP à un poste
SSH / Telnet	Accès distant	22 / 23	Contrôler un ordinateur à distance par un administrateur (avec/sans chiffrement)
SNMP	Supervision de réseau	161	Gérer et surveiller les équipements réseau (routeurs, commutateurs, …)

Schéma illustratif

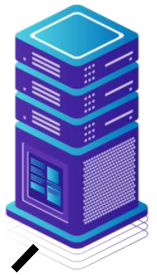
IP locale : 192.168.1.10
Adresse MAC : AA:AA:AA:AA:AA:AA



Client

Application utilisée : HTTP (port 80)
→ donc TCP est le protocole de transport.

IP locale : 10.0.0.5
Adresse MAC : EE:EE:EE:EE:EE:EE



Serveur

Interface LAN : 192.168.1.1
MAC : BB:BB:BB:BB:BB:BB
Interface WAN (vers Internet) : 102.45.60.10



Routeur Emetteur

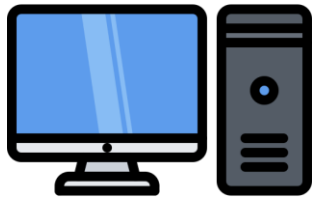


Routeur Récepteur

Interface WAN : 165.30.22.5
Interface LAN : 10.0.0.1
MAC : DD:DD:DD:DD:DD:DD

Schéma illustratif

IP locale : 192.168.1.10
Adresse MAC : AA:AA:AA:AA:AA:AA



Client

4. Application

- Le client écrit une URL dans la barre d'adresse, comme :

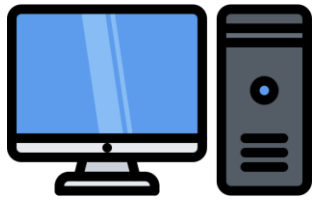
http://www.exemple.com/index.html

- Le navigateur traduit cette URL en une requête HTTP de ce type :

GET /index.html HTTP/1.1
Host: www.exemple.com

Schéma illustratif

IP locale : 192.168.1.10
Adresse MAC : AA:AA:AA:AA:AA:AA



Client

4. Application

3. Transport

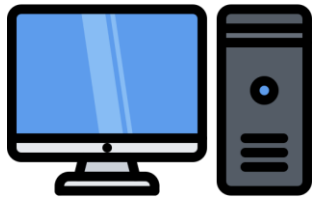
Segmentation :

- création d'un segment TCP
- Ajout du port source (ex : 50500) et du port destination (80)
- Ajout du numéro du segment

→ **Résultat :** Segment TCP

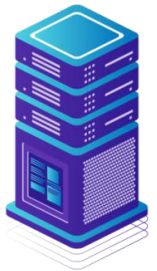
Schéma illustratif

IP locale : **192.168.1.10**
Adresse MAC : **AA:AA:AA:AA:AA:AA**



Client

IP locale : **10.0.0.5**
Adresse MAC : **EE:EE:EE:EE:EE:EE**



Serveur

4. Application

3. Transport

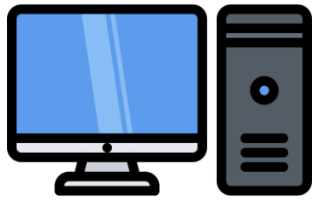
2. Internet

- Ajout de l'adresse IP source locale : 192.168.1.10
- Ajout de l'adresse IP destination : 10.0.0.5 (IP du serveur final)

→ **Résultat :** Paquet IP

Schéma illustratif

IP locale : 192.168.1.10
Adresse MAC : AA:AA:AA:AA:AA:AA



Client

Interface LAN : 192.168.1.1
MAC : BB:BB:BB:BB:BB:BB
Interface WAN (vers Internet) : 102.45.60.10



Routeur Emetteur

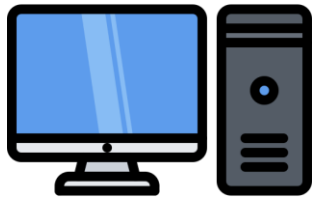
4. Application
3. Transport
2. Internet
1. Accès au réseau

- Ajout de l'adresse MAC source : AA:AA:AA:AA:AA:AA
- Ajout de l'adresse MAC destination : BB:BB:BB:BB:BB:BB (MAC du routeur)

→ **Résultat :** Trame

Schéma illustratif

IP locale : 192.168.1.10
Adresse MAC : AA:AA:AA:AA:AA:AA



Client

Interface LAN : 192.168.1.1
MAC : BB:BB:BB:BB:BB:BB
Interface WAN (vers Internet) : 102.45.60.10



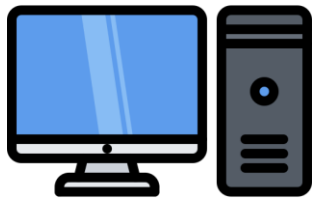
Routeur Emetteur

1. Accès au réseau

- Le routeur reçoit la trame.
- Il désencapsule (retire la trame) pour lire les adresses IP.

Schéma illustratif

IP locale : 192.168.1.10
Adresse MAC : AA:AA:AA:AA:AA:AA



Client

Routeur Emetteur

Interface LAN : 192.168.1.1
MAC : BB:BB:BB:BB:BB:BB
Interface WAN (vers Internet) : 102.45.60.10





Routeur Récepteur

Interface WAN : 165.30.22.5
Interface LAN : 10.0.0.1
MAC : DD:DD:DD:DD:DD:DD

2. Internet

1. Accès au réseau

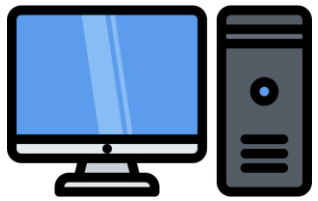
Le routeur remplace :

- Adresse IP source → 102.45.60.10 (adresse publique du routeur)
- Adresse IP destinataire → 165.30.22.5

➔ grâce au mécanisme **NAT (Network Adress Translation)**

Schéma illustratif

IP locale : 192.168.1.10
Adresse MAC : AA:AA:AA:AA:AA:AA



Client

Routeur Emetteur

Interface LAN : 192.168.1.1
MAC : BB:BB:BB:BB:BB:BB
Interface WAN (vers Internet) : 102.45.60.10





Routeur Récepteur

Interface WAN : 165.30.22.5
Interface LAN : 10.0.0.1
MAC : DD:DD:DD:DD:DD:DD

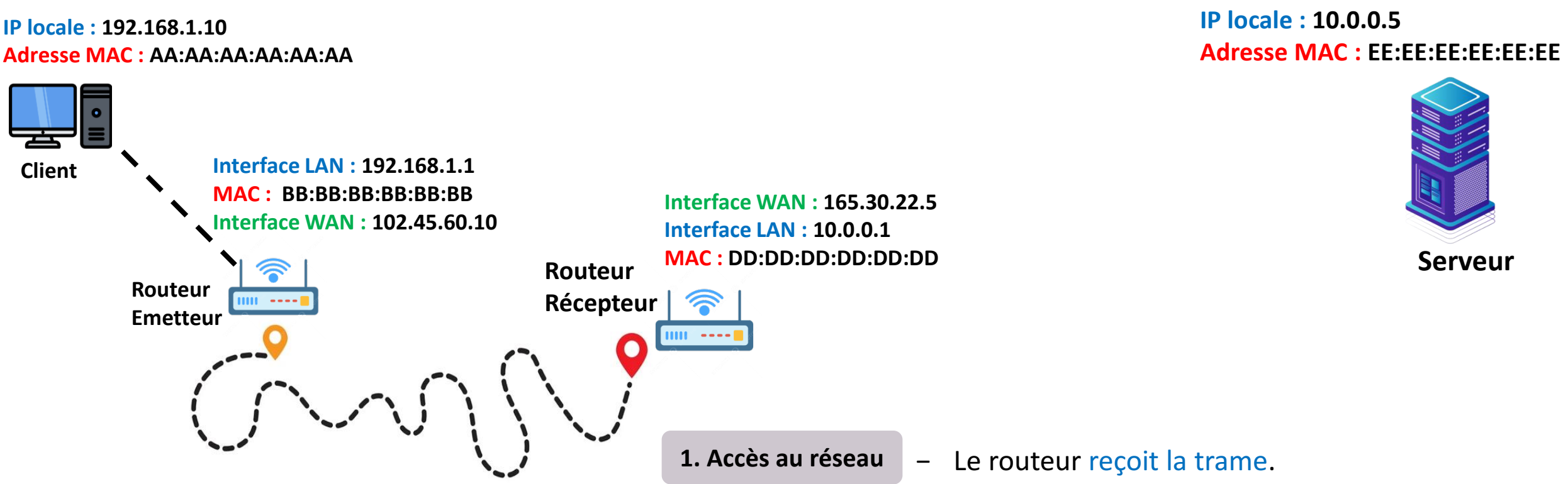
2. Internet

1. Accès au réseau

Le routeur remplace :

- Adresse MAC source → MAC du routeur (BB:BB:BB:BB:BB:BB)
- Adresse MAC destination → MAC du routeur suivant (DD:DD:DD:DD:DD:DD)

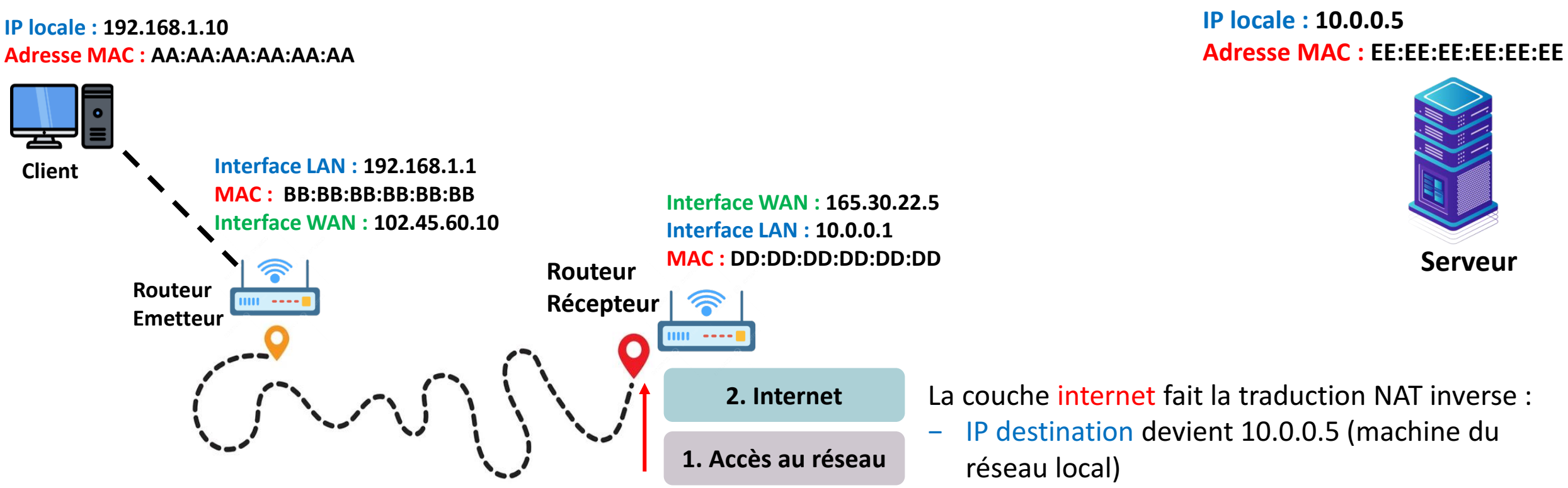
Schéma illustratif



1. Accès au réseau

- Le routeur reçoit la trame.
- Il désencapsule pour lire les adresses IP.

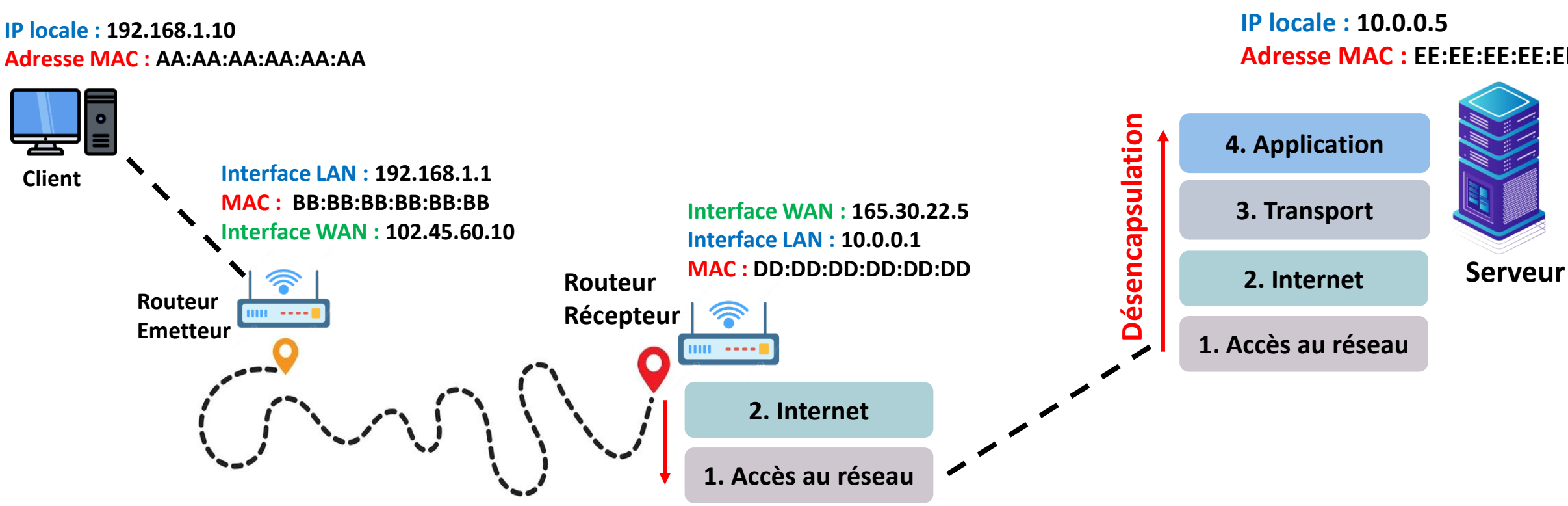
Schéma illustratif



La couche internet fait la traduction NAT inverse :

- IP destination devient 10.0.0.5 (machine du réseau local)

Schéma illustratif



La couche accès au réseau change les adresses MAC :

- MAC source : DD:DD:DD:DD:DD:DD (MAC du routeur)
- MAC destination : EE:EE:EE:EE:EE:EE (MAC du serveur)

Mécanismes de communication bas niveau

Rappel du contexte :

Quand deux applications veulent échanger des données à travers un réseau, elles doivent :

1. s'identifier (adresse IP, port),
2. établir un lien de communication fiable,
3. envoyer et recevoir des messages sous une forme compréhensible par la couche transport.

⚠ Mais ni l'application ni la couche transport ne peuvent communiquer **directement !**

C'est là qu'intervient **la socket.**



Mécanismes de communication bas niveau



- Une socket est une interface logicielle (un point d'accès) entre l'application (couche application) et le protocole de transport (TCP ou UDP).
- Elle permet à un programme d'envoyer et de recevoir des données comme s'il écrivait dans un canal de communication.
- La socket est la "porte" par laquelle les données d'une application sortent pour entrer dans le réseau (et inversement).

Un socket est le couple (adresse IP, port) → il identifie une application précise sur une machine.

→ Ce couple forme un socket endpoint unique sur une machine.

Il faut distinguer deux types principaux de sockets, selon le protocole de transport utilisé :

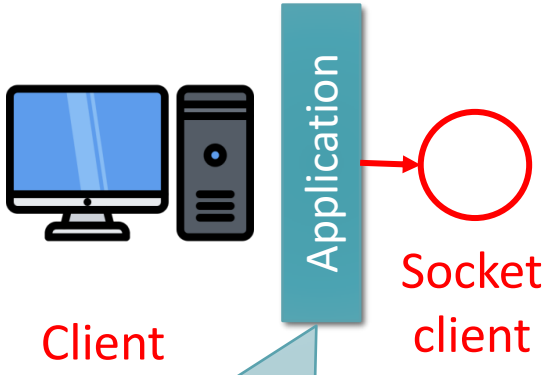
- ✓ les sockets TCP,
- ✓ et les sockets UDP, basées sur l'échange de datagrammes.

Les communications TCP ou UDP sont identifiées par le quadruplet

(IP_client, Port_client, IP_serveur, Port_serveur).

Mécanismes de communication bas niveau

Principe de base des sockets :

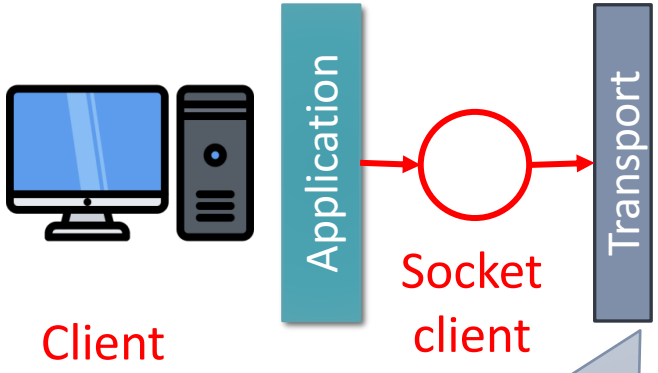


Etape 1 :
L'application (par exemple un client HTTP) veut envoyer un message. Elle crée une socket et demande à la couche transport d'envoyer les données.

Source IP : 192.168.1.15
Source Port : 53000 (port éphémère)
Destination IP : 142.250.185.78
Destination Port : 80 (HTTP)

Mécanismes de communication bas niveau

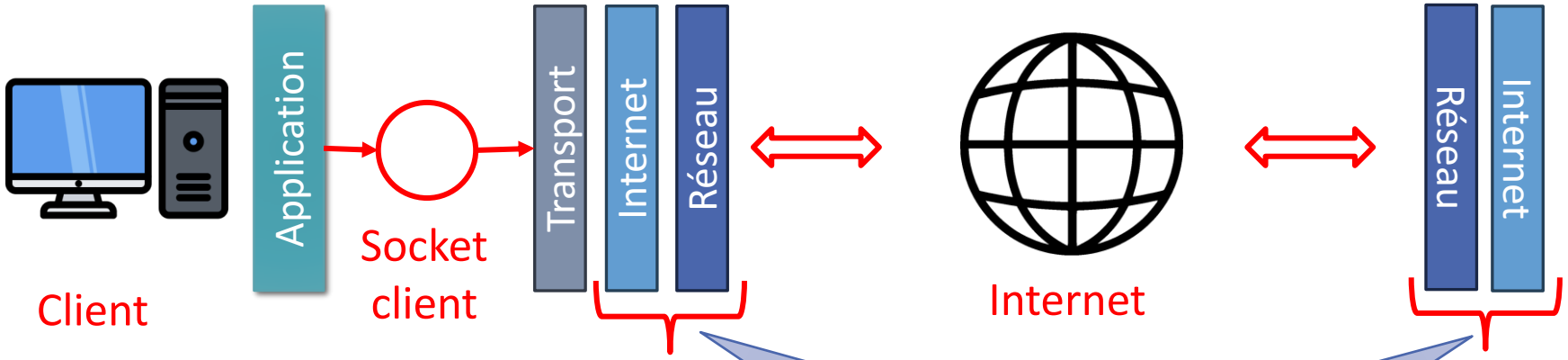
Principe de base des sockets :



Etape 2 :
La couche transport encapsule les données dans des segments, ajoute les en-têtes de contrôle et les transmet ensuite à la couche suivante.

Mécanismes de communication bas niveau

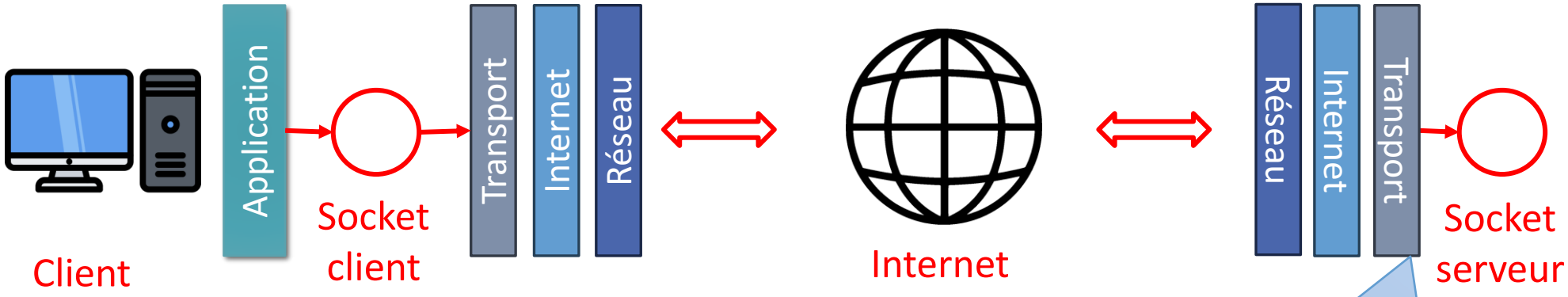
Principe de base des sockets :



Etape 3 :
Les couches internet et accès au réseau s'occupent d'acheminer les paquets jusqu'à la machine destinataire.

Mécanismes de communication bas niveau

Principe de base des sockets :

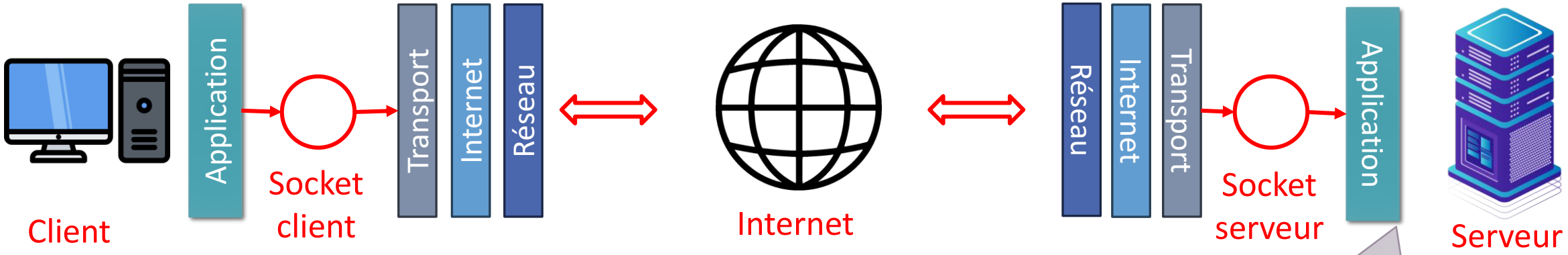


Source IP : 192.168.1.15
Source Port : 53000 (port éphémère)
Destination IP : 142.250.185.78
Destination Port : 80 (HTTP)

Etape 4 :
La couche transport du destinataire reçoit le paquet, le redirige vers la socket identifiée par le port.

Mécanismes de communication bas niveau

Principe de base des sockets :



Pourquoi les sockets sont indispensables ?

- ✓ Une application ne pourrait pas accéder au réseau,
- ✓ Les couches basses ne sauraient pas à quelle application livrer les données,
- ✓ Il n'y aurait pas de moyen simple pour établir ou maintenir une communication bidirectionnelle.

Etape 5 :
La socket transmet les données à l'application (serveur HTTP par exemple)
→ L'application lit les données depuis sa **socket**

Les sockets TCP

Le protocole TCP est orienté connexion. Avant tout échange, une connexion fiable est établie entre le client et le serveur. Les données circulent ensuite sous forme de flux continus, garantissant :

- ✓ la livraison des données sans perte,
- ✓ le respect de l'ordre d'envoi,
- ✓ et la fiabilité de la communication.

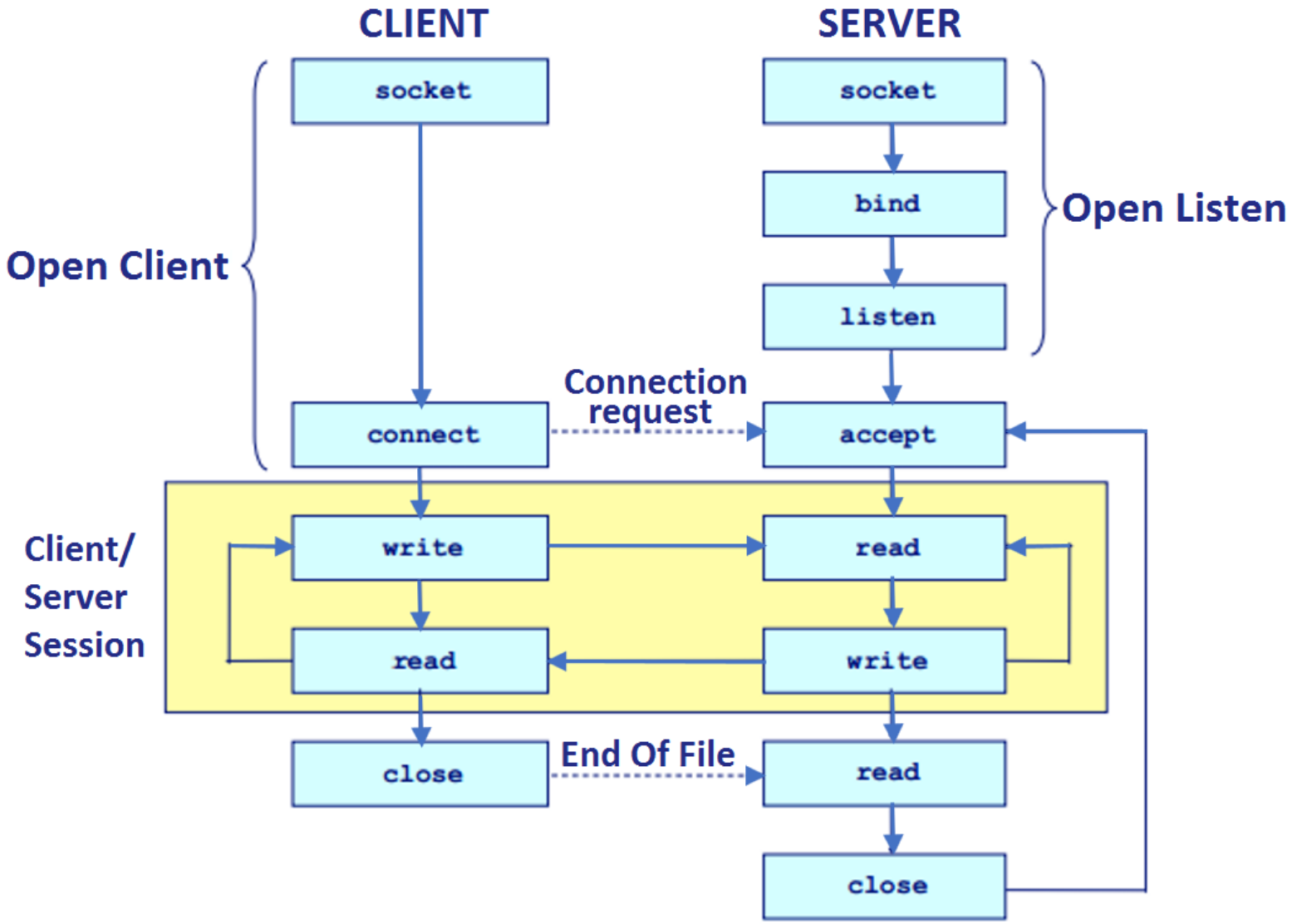
En java, les classes utilisées sont :

- ✓ ServerSocket : utilisée côté serveur pour écouter les connexions entrantes sur un port donné.
- ✓ Socket : utilisée côté client (et aussi côté serveur après acceptation) pour établir la connexion et échanger les données (lecture/écriture via des flux).

Fonctionnement :

1. Le serveur crée un objet ServerSocket et se met en attente d'un client sur un port précis.
2. Le client crée un objet Socket et tente de se connecter à ce port.
3. Une fois la connexion acceptée, les deux programmes peuvent échanger des données en utilisant des flux (InputStream et OutputStream).
4. À la fin, les sockets sont fermées.

Les sockets TCP



Les sockets UDP

Le protocole UDP est non orienté connexion. Il n'établit aucune connexion préalable entre le client et le serveur. Les données sont envoyées sous forme de datagrammes indépendants : chaque paquet circule séparément dans le réseau, sans garantie d'ordre ni de réception.

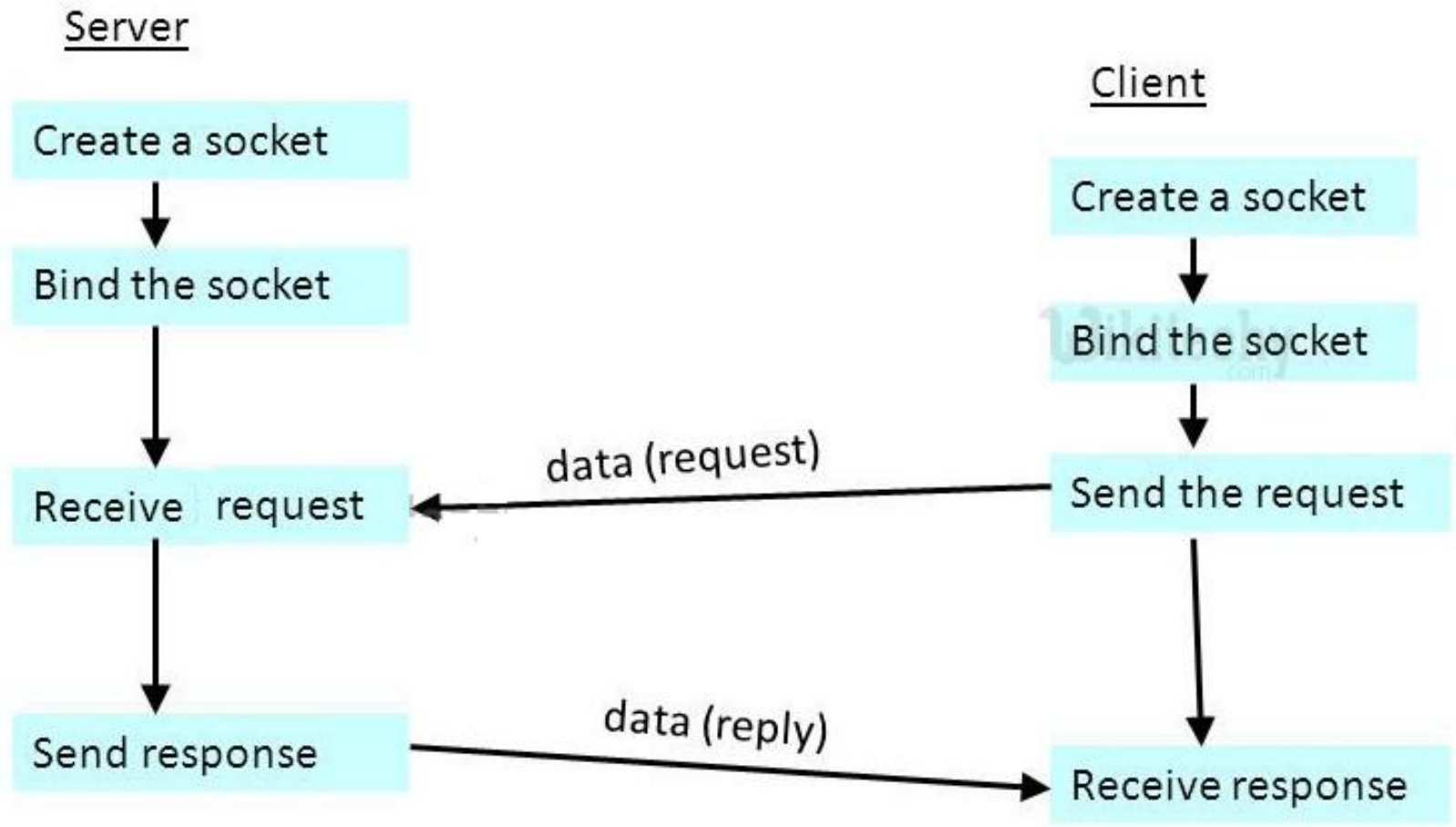
En java, les classes utilisées sont :

- ✓ DatagramSocket : représente la « prise » de communication pour envoyer ou recevoir des datagrammes.
- ✓ DatagramPacket : représente le paquet de données à transmettre (il contient les données, l'adresse IP du destinataire et le port).

Fonctionnement :

1. Le serveur UDP crée un DatagramSocket et attend la réception d'un paquet.
2. Le client UDP crée également un DatagramSocket et envoie un DatagramPacket vers l'adresse et le port du serveur.
3. Chaque message envoyé est indépendant des autres → il n'existe aucune session ni connexion persistante.
4. Après réception, le serveur peut répondre via un autre datagramme.

Les sockets UDP



Bas niveau VS. Haut niveau

Rappel : mécanismes de communication bas niveau (les sockets)

Le programmeur gère :
- la création de la connexion (adresse IP, port) ;
- l'envoi/réception de flux d'octets ;
- le format des messages (encodage, protocole applicatif).

Avantage : contrôle total, performance.
Inconvénient : code complexe, risque d'erreurs, couplage fort entre client et serveur.

Avec la complexité croissante des applications distribuées, les développeurs ont cherché à :
- ✓ Simplifier la programmation réseau,
- ✓ Automatiser la gestion des échanges (marshalling, transport, erreurs),
- ✓ Rendre la communication transparente (comme un appel local),
- ✓ Assurer l'interopérabilité entre systèmes et langages.

➡ D'où la naissance des **mécanismes de communication haut niveau**.

Bas niveau VS. Haut niveau

Aspect	Bas niveau (Sockets)	Haut niveau (RPC, RMI, ...)
Niveau d'abstraction	Très bas : Le programmeur manipule les paquets, adresses, ports, flux.	Élevé : Le programmeur appelle simplement une fonction ou méthode distante.
Unité d'échange	Flux d'octets	Appel de fonction / méthode / service
Gestion du protocole	Manuelle	Automatisée
Format des données	Défini par le programmeur	Structuré (JSON, XML, Protobuf...)
Complexité du code	Élevée (il faut gérer la communication réseau manuellement).	Réduite (les détails réseau sont cachés).
Contrôle sur les détails réseau	Total : La communication réseau est explicite.	Transparence partielle : on a l'impression d'un appel local.
Objectif	Performance, flexibilité (Le programmeur contrôle tout)	Productivité (facilité et la rapidité de développement), simplicité, concentration sur la logique métier, pas sur la communication.

Historique et évolution

Époque	Contexte technique et besoins	Mécanismes dominants	Caractéristiques principales
Années 1970 – début 1980	Début des réseaux informatiques : besoin d'envoyer des données entre machines.	Sockets	Mécanisme bas niveau : ✓ Communication via des flux d'octets. ✓ Programmation manuelle (adresses IP, ports, protocole TCP/UDP). ✓ Très flexible mais complexe.
Milieu – fin des années 1980	Complexité de programmation avec sockets : les développeurs veulent rendre la communication transparente.	RPC (Remote Procedure Call) (Sun RPC, ONC RPC)	Première abstraction haut niveau : ✓ appel de procédure distante. ✓ Sérialisation automatique des arguments. ✓ Masque le transport réseau.
Années 1990	Montée de la programmation orientée objet → besoin d'appels de méthodes sur des objets distants.	CORBA, Java RMI, DCOM	Extension du principe RPC aux objets distants: ✓ Gestion des références d'objets et des exceptions à distance. ✓ Communication entre langages et plateformes différentes (CORBA).

Historique et évolution

Époque	Contexte technique et besoins	Mécanismes dominants	Caractéristiques principales
Années 2000	Explosion du Web → besoin d'interopérabilité et de communication entre applications hétérogènes.	SOAP, XML-RPC, Web Services	✓ Communication via **HTTP + XML**. ✓ Standardisation (WSDL, UDDI). ✓ Largement utilisé dans les entreprises et systèmes interconnectés.
Années 2010	Généralisation du Web, besoin de légèreté et de scalabilité.	REST, JSON-RPC, gRPC, Thrift	✓ Communication via HTTP + JSON (REST) ou via binaire (gRPC, Thrift). ✓ APIs simples, typées et performantes. ✓ Adaptées aux microservices et aux applications cloud.
Années 2020+	Environnements cloud, mobiles et temps réel.	GraphQL, WebSockets, Async APIs	✓ Communication flexible et dynamique. ✓ Support du streaming, de la communication bidirectionnelle. ✓ Forte orientation vers la performance et la résilience.

Mécanismes de communication haut niveau

Un mécanisme de communication haut niveau est une abstraction logicielle qui permet à des composants répartis de communiquer sans gérer directement les détails bas niveau du réseau (sockets, ports, formats binaires, etc.).

C'est une abstraction qui masque la complexité des communications réseau en fournissant des modèles de programmation plus intuitifs et plus proches des concepts métier. → Le programmeur interagit via des objets, des procédures, des services, ou des messages structurés au lieu de manipuler des flux d'octets.

Les grandes familles de mécanismes haut niveau :

RPC (Remote Procedure Call)

- appeler une fonction distante comme une fonction locale.

RMI (Remote Method Invocation)

- Extension orientée objet du RPC

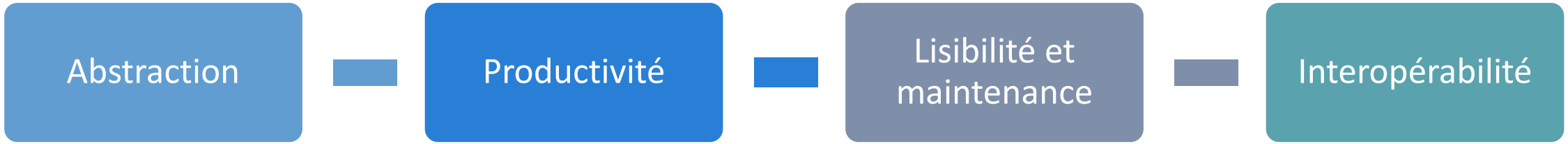
Services Web (SOAP, REST, gRPC)

Mécanismes orientés message (communication asynchrone) : Basés sur des files de messages ou des brokers (ex. JMS, RabbitMQ, Kafka).

RPC (Remote Procedure Call)

Les RPC apparaissent comme une abstraction qui permet au développeur d'appeler une procédure (ou méthode) située sur une autre machine comme s'il l'appelait localement. L'objectif est de masquer la complexité du réseau et de rendre la programmation distribuée plus proche de la programmation locale.

Pourquoi RPC ?



Principe de base du RPC :

1. Le client appelle une fonction "comme d'habitude".
2. Le système RPC prépare l'appel et l'envoie au serveur.
3. Le serveur reçoit la demande, exécute la fonction demandée.
4. Le résultat est renvoyé automatiquement au client.
5. Le client récupère le résultat comme s'il venait d'une fonction locale.

Composants clés et vocabulaire

Stub client / Stub Serveur → Les intermédiaires qui transmettent l'appel

- ✓ Stub client (proxy) : code généré côté client qui remplace l'appel réel et s'occupe du marshalling + envoi.
- ✓ Skeleton (stub serveur) : code côté serveur qui reçoit les messages, fait l'unmarshalling et appelle la procédure réelle.

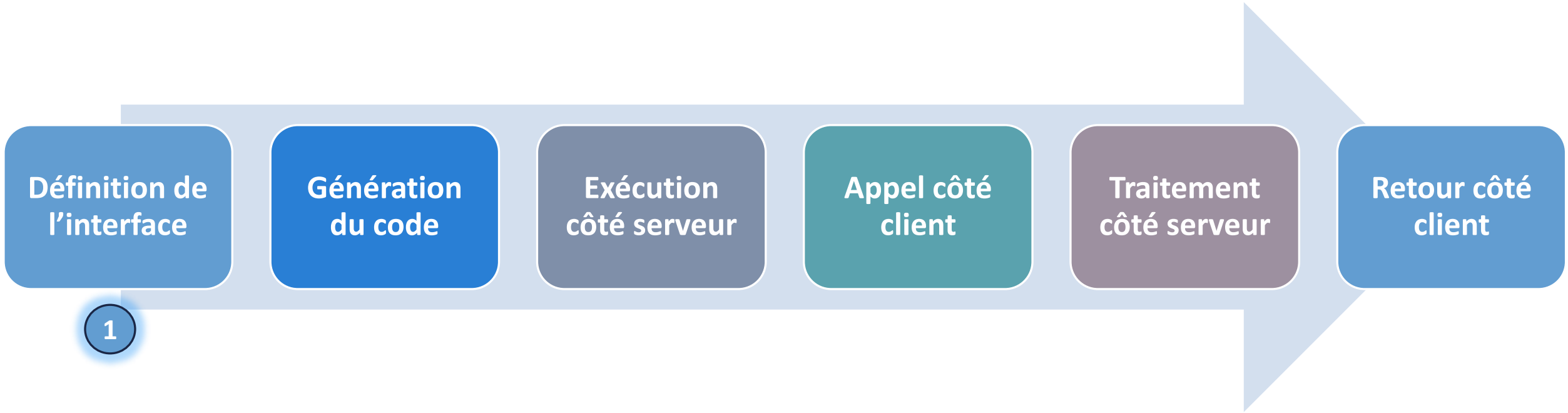
Marshalling / Unmarshalling → Conversion des données à envoyer et à recevoir

- ✓ Marshalling / Serialization : conversion des paramètres en une représentation binaire (Protobuf, Thrift) ou textuelle (JSON, XML) pour transport.
- ✓ Unmarshalling / Deserialization : reconstitution des paramètres à l'autre bout.

IDL (Interface Definition Language) → Langage de description de l'interface

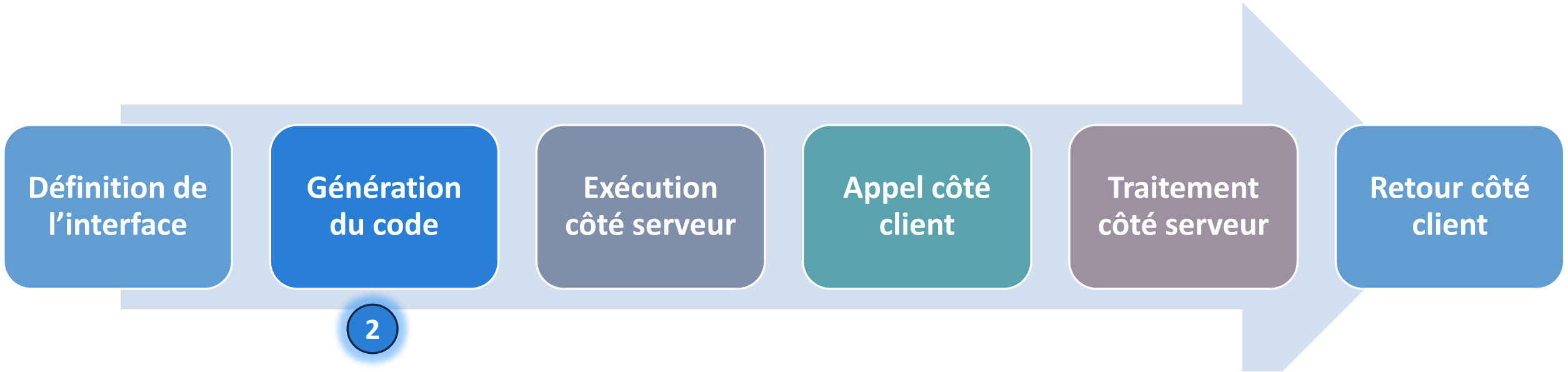
C'est un fichier qui décrit les fonctions accessibles à distance (leurs noms, leurs paramètres, leurs types de retour, etc.). → C'est le développeur du service (serveur) qui écrit le fichier IDL.

Fonctionnement détaillé du RPC



La signature de la fonction distante (types d'arguments, types de retour) est écrite dans l'IDL.

Fonctionnement détaillé du RPC

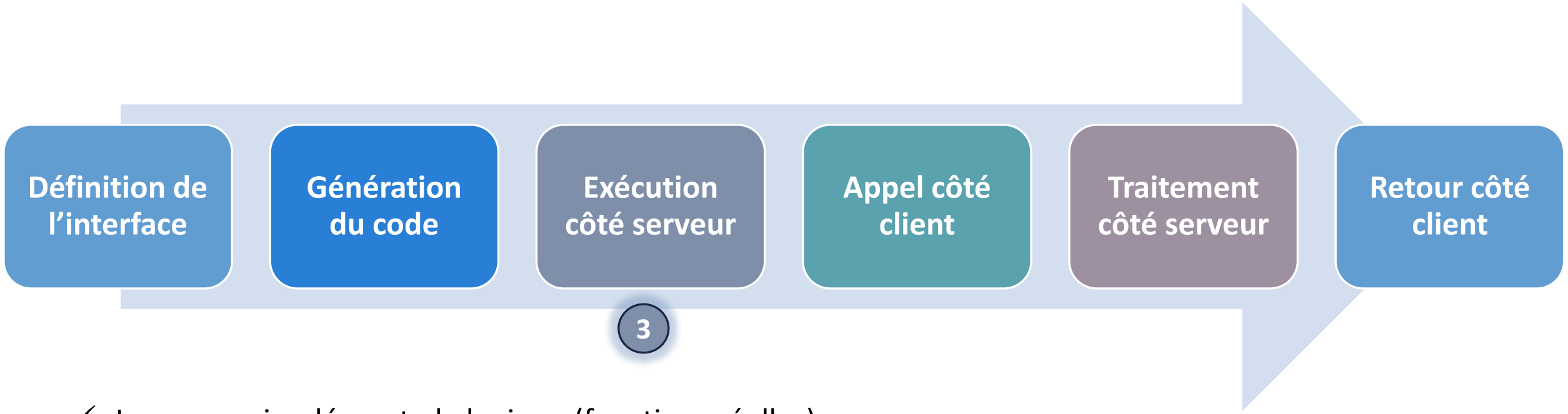


Des outils spéciaux (qui dépendent de la technologie RPC) lisent le fichier d'interface et génèrent automatiquement au moment de la compilation ou du déploiement :

- ✓ un stub côté client (qui remplace la fonction distante),
- ✓ un squelette côté serveur (qui reçoit la requête et appelle la vraie fonction).

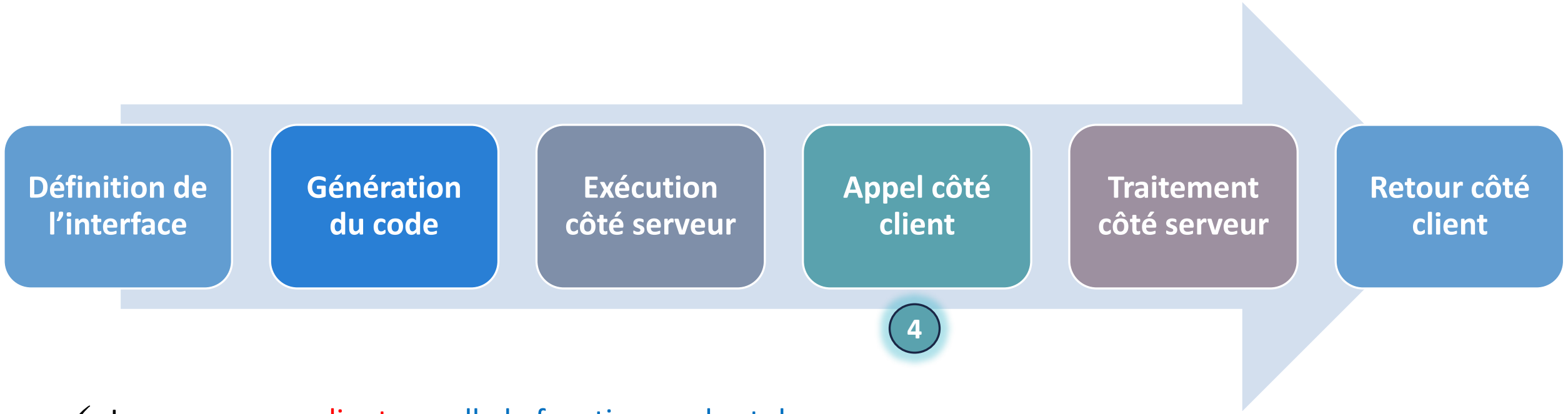
Technologie RPC	Outil de génération
ONC RPC (Sun RPC)	rpcgen
DCE RPC (Distributed Computing Environment)	idl (compilateur IDL DCE)
CORBA (Common Object Request Broker Architecture)	idl compiler (souvent idltojava, omniidl, etc.)
Microsoft RPC (Windows)	MIDL (Microsoft IDL Compiler)

Fonctionnement détaillé du RPC



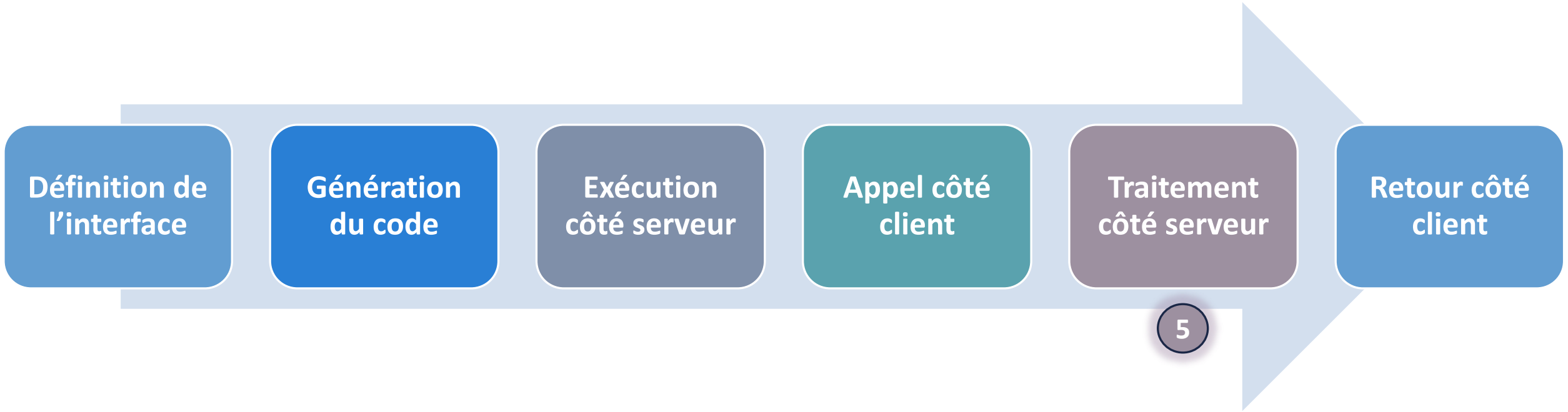
- ✓ Le serveur implémente la logique (fonctions réelles).
- ✓ Il enregistre son service dans un annuaire (optionnel) ou écoute sur un port connu.

Fonctionnement détaillé du RPC



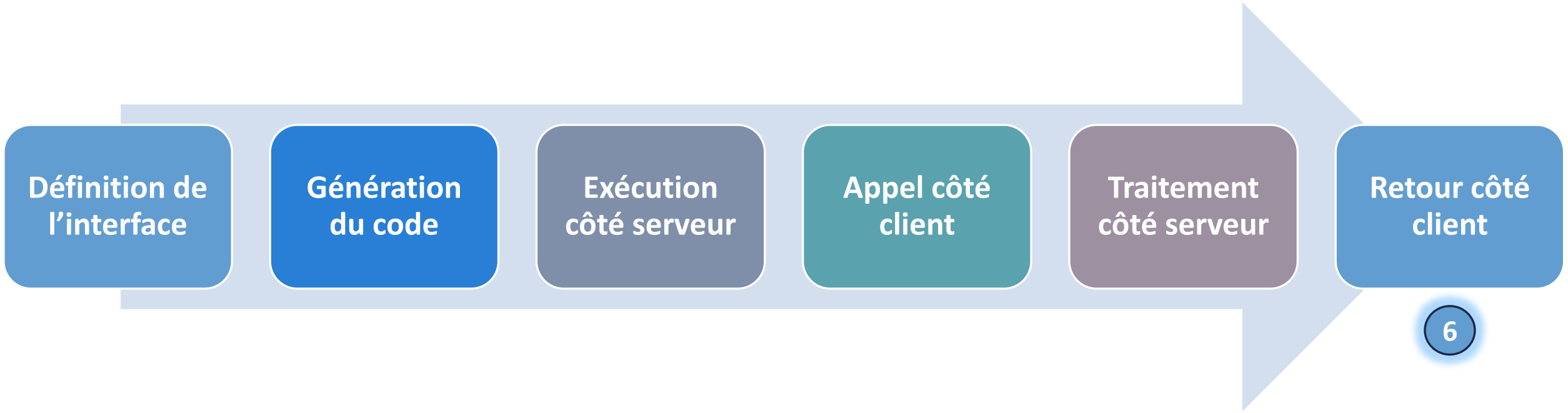
- ✓ Le programme client appelle la fonction sur le stub.
- ✓ Le stub exécute le marshalling : transforme les arguments en message.
- ✓ Le message est envoyé via le transport (socket TCP).
- ✓ Le stub attend une réponse (s'il s'agit d'un RPC synchrone).

Fonctionnement détaillé du RPC



- ✓ Le skeleton reçoit le message, effectue l'unmarshalling.
- ✓ Il appelle la fonction réelle.
- ✓ Il récupère le résultat (ou gère l'exception).
- ✓ Il envoie la réponse marshallée.

Fonctionnement détaillé du RPC



Le stub client reçoit la réponse, effectue l'unmarshalling et retourne la valeur au programme appelant.

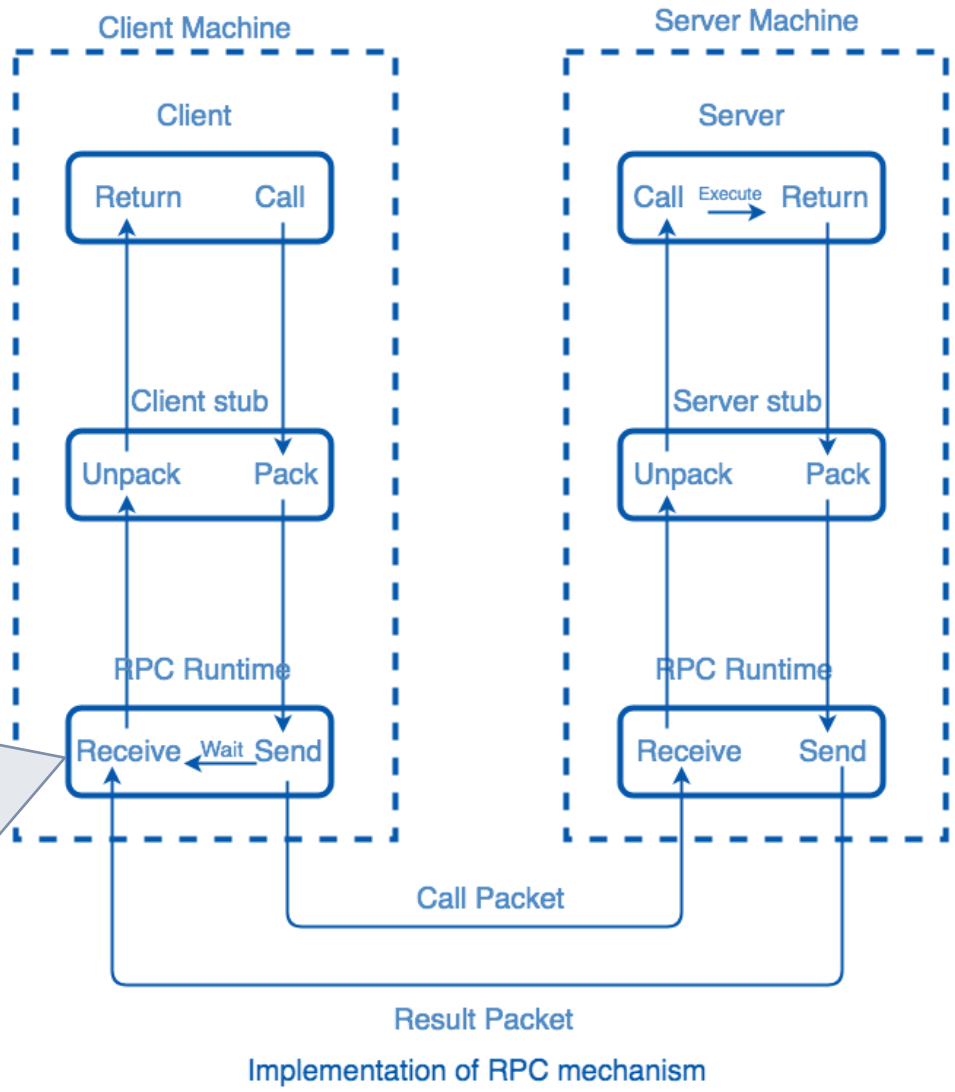
Fonctionnement détaillé du RPC

RPC runtime : C'est le mécanisme sous-jacent de communication.

→ Une couche logicielle interne fournie par la bibliothèque RPC

Il envoie les messages via le réseau (souvent avec des sockets TCP/UDP).

Il gère la transmission, l'attente de réponse, et la réception.



Implementation of RPC mechanism

Question de reflexion

Comment le client « sait » que la fonction existe ?

Il y a plusieurs cas possibles selon la technologie utilisée :

Contexte	Comment le client découvre le service ?
Même entreprise / même projet	Le développeur reçoit le fichier IDL ou l'interface Java. C'est souvent documenté en interne.
Environnement distribué (ex : CORBA, RMI Registry, gRPC)	Le client interroge un annuaire de services (naming service). Cet annuaire dit sur quelle machine/port se trouve le service.
Web moderne (SOAP, REST, gRPC)	Il existe une description publique (ex : fichier WSDL ou .proto) que le client télécharge pour générer son stub automatiquement.

Pseudocode RPC synchrone (vue logique)

IDL (Interface Definition Language)

```
service Calculator {
    int add(int a, int b);
    int sub(int a, int b);
}
```

Implémentation réelle du service coté serveur

```
public class CalculatorImpl implements Calculator {
    @Override
    public int add(int a, int b) {
        System.out.println("Reçu une requête RPC: add(" + a + ", " + b + ")");
        return a + b; // traitement réel
    }
    // méthode sub
}
```

Enregistrement du service par le serveur auprès du système RPC

```
public static void main(String[] args) {
    Calculator service = new CalculatorImpl();
    registerService("Calculator", service); // rendu disponible via RPC
    System.out.println("Service Calculator prêt.");
}
```

Pseudocode RPC synchrone (vue logique)

Skeleton coté serveur

```
public class CalculatorServerStub {
   private Calculator impl;
   // référence vers l'implémentation réelle

   public CalculatorServerStub(Calculator impl) {
      this.impl = impl;
   }

   // appelé quand une requête RPC arrive
   public String handleRequest(String requestJson) {
      // Exemple : requestJson =
{"method":"add","args":[3,4]}
      JSONObject req = new JSONObject(requestJson);
      String method = req.getString("method");
      JSONArray args = req.getJSONArray("args");
      int result = 0;
      switch (method) {
         case "add":
            result = impl.add(args.getInt(0), args.getInt(1));
            break;
         case "sub":
            result = impl.sub(args.getInt(0), args.getInt(1));
            break;
         default:
            throw new RuntimeException("Méthode
inconnue: " + method);
      }

      // renvoyer la réponse JSON
      return "{\"result\": " + result + "}";
   }
}
```

Pseudocode RPC synchrone (vue logique)

Stub coté client (proxy)

```
public class CalculatorClientStub implements Calculator {
   private NetworkChannel channel;
   // abstraction de la connexion réseau

   public CalculatorClientStub(NetworkChannel channel) {
      this.channel = channel;
   }
   public int add(int a, int b) {
      String req = "{\"method\":\"add\",\"args\":[" + a + ","
+ b + "]}";
      String resp = channel.sendRequest(req); // envoie au
serveur
      JSONObject json = new JSONObject(resp);
      return json.getInt("result");
   }
```

```
  public int sub(int a, int b) {
      String req = "{\"method\":\"sub\",\"args\":[" + a + ","
+ b + "]}";
      String resp = channel.sendRequest(req);
      JSONObject json = new JSONObject(resp);
      return json.getInt("result");
   }
}
```

Pseudocode RPC synchrone (vue logique)

Appel coté client

```
calculator remoteService = getRemoteService("Calculator");
result = remoteService.add(3, 4)   // semble local
print(result)  // 7
```

Ce qui se produit en coulisse :

1. stub.client.marshall(3,4) → message
2. transport.send(message)
3. server.receive → skeleton.unmarshall → server.add(3,4) → 7
4. server.marshall(7) → transport.send
5. client.receive → stub.unmarshall → retourne 7

RPC runtime

```
public class RpcRuntime {
   public static Object getRemoteService(String name) {
      if (name.equals("Calculator")) {
         return new CalculatorClientStub(new
NetworkChannel("localhost", 5000));
      }
      throw new RuntimeException("Service inconnu : " +
name);
   }
}
```

Avantages du RPC

Facilite le développement

- Code plus proche du modèle local.

Encapsulation de la communication

- Développeur focalisé sur l'API métier

IDL + génération de code

- Cohérence des interfaces, moins d'erreurs manuelles.

Support de différents paradigmes

- sync/asych, streaming

Interopérabilité

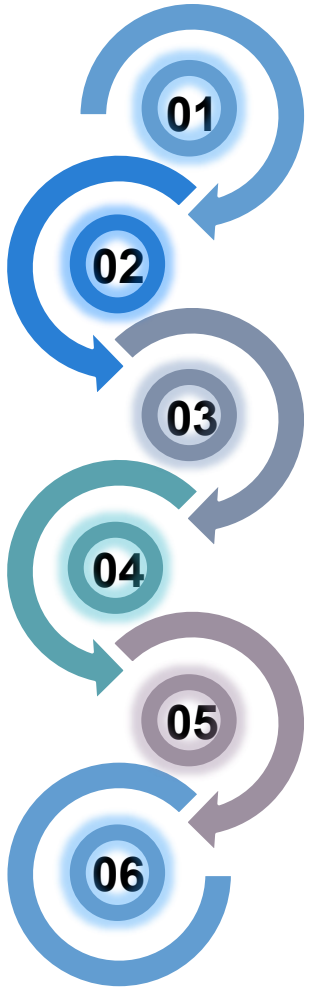
- Avec SOAP/gRPC, on peut connecter des services écrits dans différents langages.

Inconvénients du RPC

Sérialisation coûteuse : conversion/validation des objets peut être lourde.

Scalabilité : un nombre élevé d'appels RPC synchrones peut créer des goulots d'étranglement.

Debug et traçabilité : plus difficile que pour des appels locaux — nécessité d'outils de tracing distribué.



Fausse impression de transparence : un appel distant n'est pas un appel local — latence, pannes, variabilité.

Couplage sémantique : changements d'interface peuvent casser les clients si versioning absent.

Sécurité : exposition de services via le réseau nécessite authentification, chiffrement, contrôle d'accès.

Transition historique : RPC → CORBA → RMI

Lorsque le RPC (Remote Procedure Call) a été inventé dans les années 1980, la majorité des langages utilisés (comme C, Pascal, Ada) étaient procéduraux.

➡ On appelait donc à distance des fonctions ou procédures, pas des objets.

Mais à la fin des années 1980 et au début des années 1990, la programmation orientée objet (POO) s'est imposée (C++, Smalltalk, puis Java).

➡ Les systèmes distribués devaient eux aussi évoluer pour permettre l'appel de méthodes sur des objets distants, pas seulement de fonctions.

Avant que Java n'existe (1995), l'OMG (Object Management Group) a conçu un standard appelé :
CORBA – Common Object Request Broker Architecture (début des années 1990).

Objectif de CORBA :

Permettre à des programmes écrits dans différents langages (C++, etc.) de communiquer entre eux, en appelant des méthodes d'objets distants de manière transparente.

Transition historique : RPC → CORBA → RMI

CORBA est une extension orientée objet du RPC :

1. Définition des interfaces dans un langage neutre appelé IDL (Interface Definition Language).
2. Compilation de l'IDL pour générer automatiquement les stubs (côté client) et les squelettes (côté serveur).
3. Le client appelle une méthode sur l'objet distant via le stub, comme s'il s'agissait d'un objet local.
4. L'ORB (Object Request Broker) se charge de transporter la requête au serveur et de gérer la communication réseau.
5. Le serveur reçoit l'appel via le squelette, exécute la méthode et renvoie le résultat au client à travers l'ORB.

L'ORB joue exactement le même rôle que le RPC Runtime dans un système RPC classique. il est l'intermédiaire qui s'occupe de toute la mécanique réseau entre le client et le serveur.

Limites de CORBA :

CORBA a eu un grand succès dans les années 1990, étant donné qu'il visait une interopérabilité multi-langages, mais :

- ✓ Il était complexe à configurer ;
- ✓ Il nécessitait beaucoup de fichiers et de couches pour faire de simples appels.

La naissance de RMI (Remote Method Invocation)

Quand Java est apparu (vers 1995), il a apporté un modèle objet simple et uniforme, et une machine virtuelle (JVM) garantissant la portabilité.

Les ingénieurs de Sun Microsystems se sont inspirés du modèle CORBA, mais ont voulu un système :

- ✓ Plus léger,
- ✓ Spécifique à Java,
- ✓ Utilisant la sérialisation native des objets Java.

⇒ C'est ainsi qu'est né RMI (Remote Method Invocation)

RMI : Un mécanisme purement Java, inspiré de CORBA, qui permet d'invoquer une méthode sur un objet situé sur une autre machine Java, comme si l'objet était local.

Le but du RMI est donc :

- ✓ de permettre la communication entre objets Java distribués sur un réseau,
- ✓ tout en conservant les principes de la programmation orientée objet (encapsulation, héritage, polymorphisme…),
- ✓ et en cachant complètement la complexité du réseau au développeur.

Les composants RMI

Client

Programme qui invoque les méthodes distantes à travers une référence obtenue via le registre RMI.

Serveur

Héberge les objets distants et les enregistre dans le registre RMI.

Registre RMI

Service de nommage qui associe un nom logique à un objet distant pour permettre au client de le localiser.

Classe d'implémentation

Contient le code de ces méthodes. Elle étend UnicastRemoteObject.

Rend un objet disponible et accessible à distance

Interface distante

Définit les méthodes accessibles à distance. Elle hérite de java.rmi.Remote.

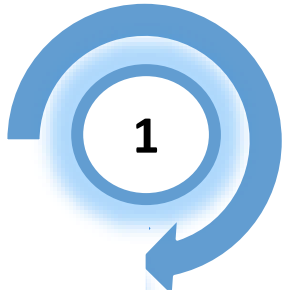
Marqueur : Pour indiquer que les méthodes qu'elle définit peuvent être appelées à distance

Stub client

Définit les méthodes accessibles à distance. Elle hérite de java.rmi.Remote.

*** Skeleton serveur :** Initialement, le compilateur RMI (RMIC) générait stubs et skeletons via compilation, mais le skeleton, depuis java 5, est créé dynamiquement sans nécessiter de compilation manuelle

Etapes de fonctionnement RMI



Étape 1 : Définir l'interface distante

L'interface distante définit les méthodes qui peuvent être appelées depuis une autre JVM.
Elle doit :

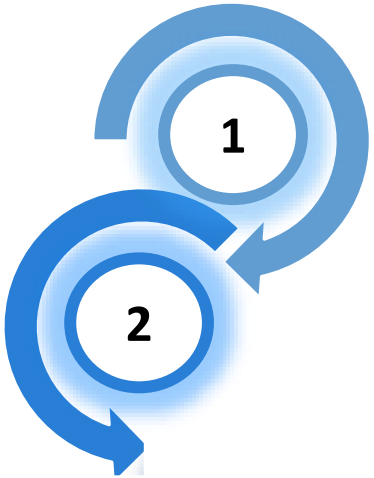
- ✓ étendre java.rmi.Remote
- ✓ déclarer toutes ses méthodes avec throws RemoteException.

Exemple : Code Interface distante : **Hello.java**

```
import java.rmi.Remote;
import java.rmi.RemoteException;

// Interface distante
public interface Hello extends Remote {
   String direBonjour(String nom) throws RemoteException;
}
```

Etapes de fonctionnement RMI



Étape 2 : Créer la classe d'implémentation distante

Cette classe :
- ✓ implémente l'interface distante,
- ✓ hérite de UnicastRemoteObject,
- ✓ fournit le code concret de chaque méthode distante.

Exemple : Code Implémentation : **HelloImpl.java**

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

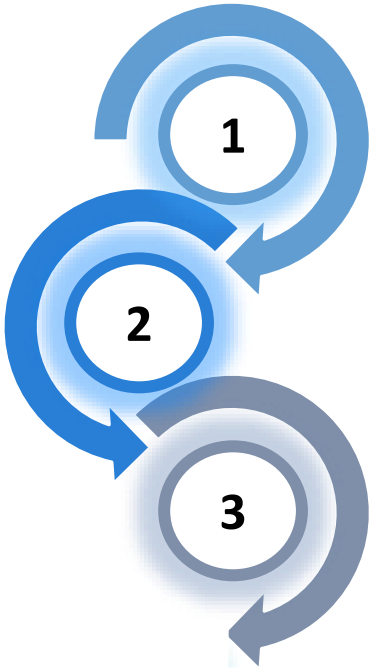
public class HelloImpl extends UnicastRemoteObject implements Hello {
    public HelloImpl() throws RemoteException {
        super();
    }
    @Override
    public String direBonjour(String nom) throws RemoteException {
        return "Bonjour, " + nom + " !";
    }
}
```

Classe qui implémente l'interface distante

Constructeur qui peut lever une RemoteException

Définition de la méthode distante

Etapes de fonctionnement RMI



Étape 3 : Mettre en place le serveur RMI

Le serveur :
- ✓ crée et exporte l'objet distant,
- ✓ démarre le **registre RMI** (service de nommage),
- ✓ enregistre l'objet dans le registre sous un nom logique (via Naming.rebind).

Exemple : Code serveur : **ServeurRMI.java**

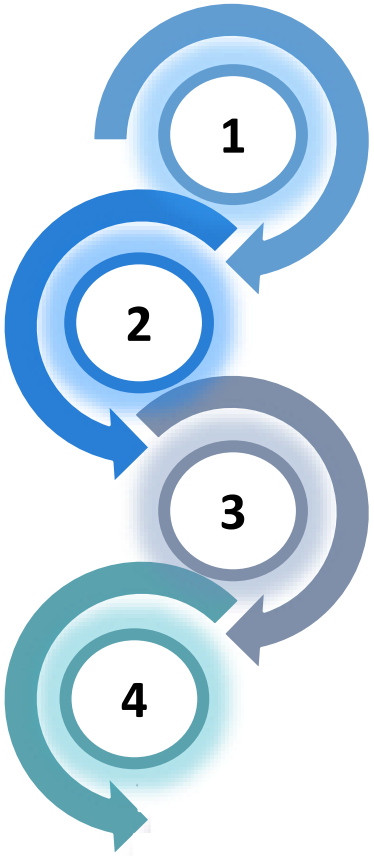
```
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;
public class ServeurRMI {
   public static void main(String[] args) {
      try {
         LocateRegistry.createRegistry(1099);
         HelloImpl obj = new HelloImpl();
         Naming.rebind("rmi://localhost/HelloService", obj);
         System.out.println("Serveur RMI prêt et en attente
des requêtes...");
      } catch (Exception e) {  e.printStackTrace();  }    }    }
```

Démarrage du registre RMI souvent sur le port 1099

Création de l'objet distant

Enregistrement de l'objet dans le registre RMI

Etapes de fonctionnement RMI



Étape 4 : Ecrire le client RMI

Le client :
1. localise l'objet distant via le registre RMI (Naming.lookup()),
2. obtient une référence distante (le stub),
3. appelle les méthodes distantes comme si elles étaient locales.

Exemple : Code client : **ClientRMI.java**

```
import java.rmi.Naming;

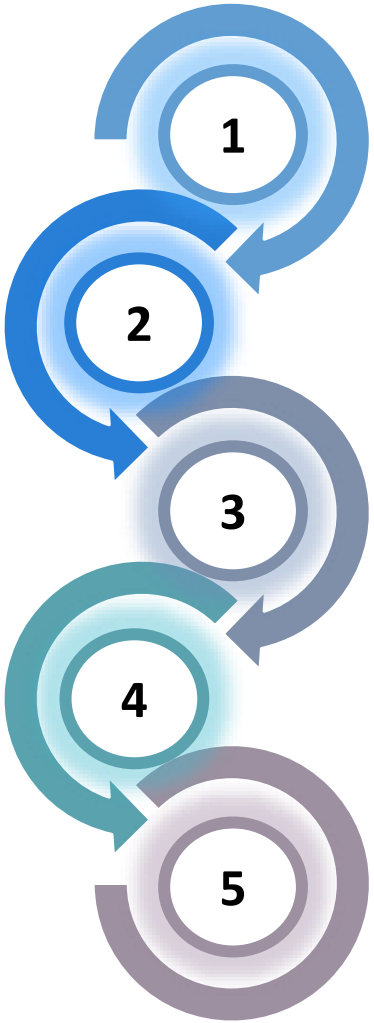
public class ClientRMI {
   public static void main(String[] args) {
      try {
         Hello obj = (Hello) Naming.lookup("rmi://localhost/HelloService");
         String reponse = obj.direBonjour("Mouna");
         System.out.println("Réponse du serveur : " + reponse);
      } catch (Exception e) { e.printStackTrace(); }
   }
}
```

Localisation de l'objet distant

Invocation de la méthode distante

Affichage du résultat retourné par le serveur

Etapes de fonctionnement RMI



Étape 5 : Compilation et exécution

1. Compiler tous les fichiers Java : `javac *.java`

2. Démarrer le registre RMI (dans un terminal séparé) : `rmiregistry`

   ➔ Ce service écoute par défaut sur le port 1099.
   ➔ Il faut le lancer dans le même répertoire que les classes compilées.

3. Démarrer le serveur : `java ServeurRMI`

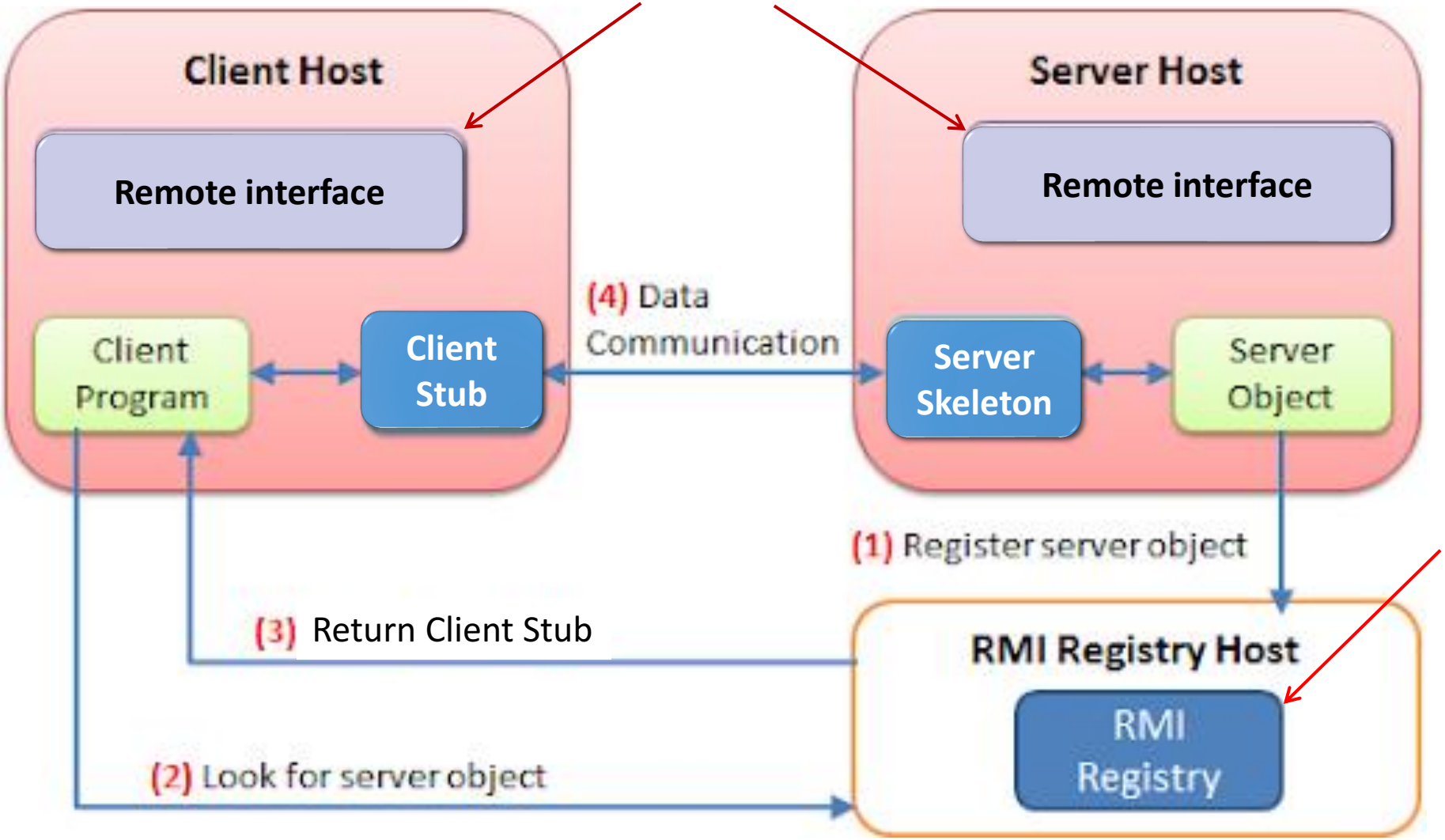
   → Message attendu : Serveur RMI prêt et en attente des requêtes...

4. Lancer le client : `java ClientRMI`

   → Réponse du serveur : Bonjour, Mouna !

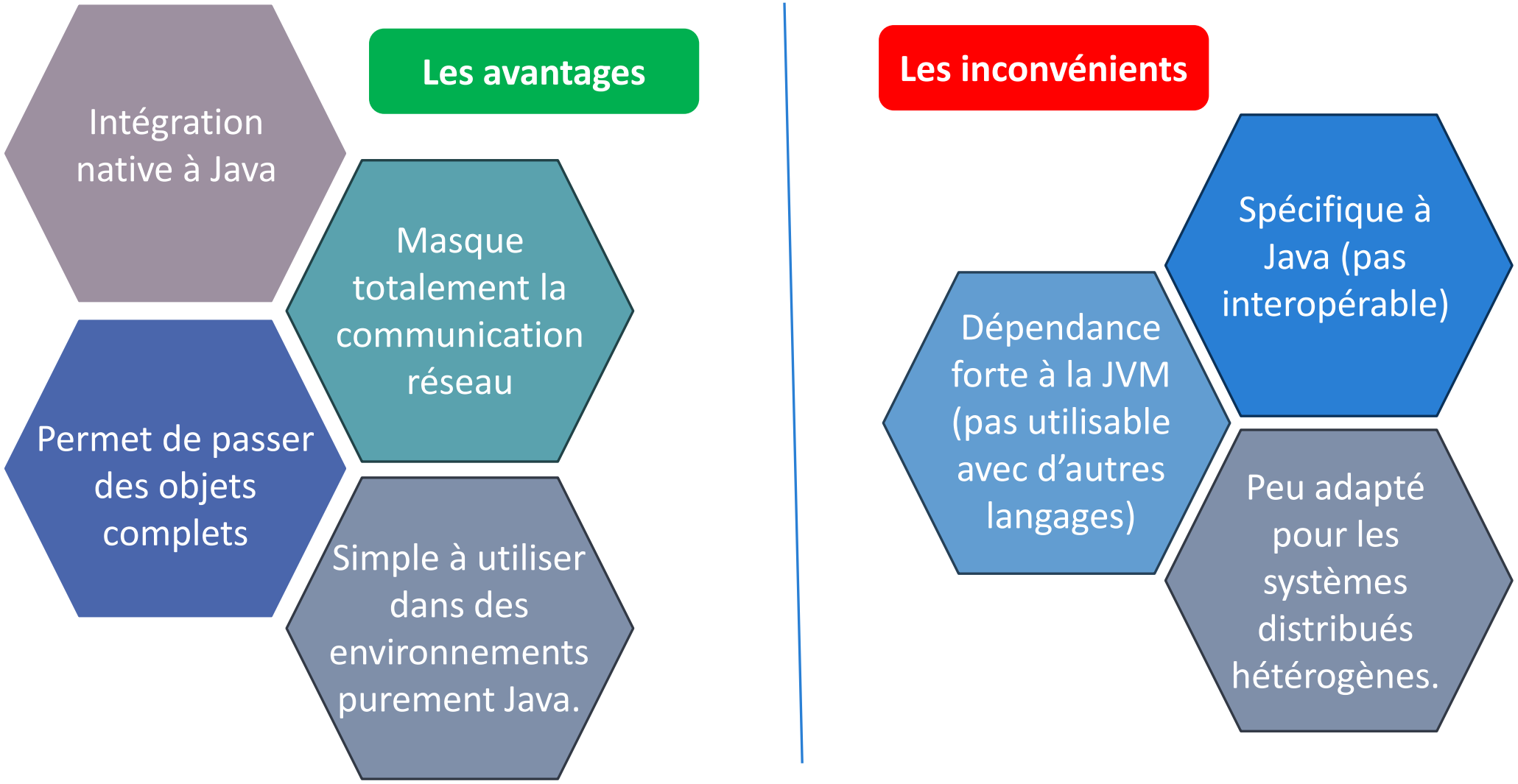
L'architecture générale RMI

définit le contrat entre client et serveur (la méthode que le client peut appeler).



Registre des objets distants : stocke les références d'objets distants exportés

Avantages et inconvénients du RMI



RPC vs. RMI

Aspect	RPC	RMI
Paradigme	Procédural	Orienté Objet
Langage	Multi-langages (C, C++, etc.)	Spécifique à Java
Ce qu'on appelle	Des fonctions (procédures)	Des méthodes d'objets distants
Type de données transmis	Types simples (int, string, etc.)	Objets Java (via sérialisation)
Interface	Définie via un fichier IDL	Interface Java qui étend Remote
Interopérabilité	Forte (langages différents possibles)	Limitée à Java
Outil de génération	idlj, protoc, etc.	rmic
Squelette	Toujours présent	Automatique depuis Java 5